

Les Entrées Sorties en java

Réalisé par Mme Gatri.latifa

Les Entrées Sorties en java

Entrée/sortie : échange de données entre le programme et une source :

- entrée : au clavier, lecture d'un fichier, communication réseau
- sortie : sur la console, écriture d'un fichier, envoi sur le réseau

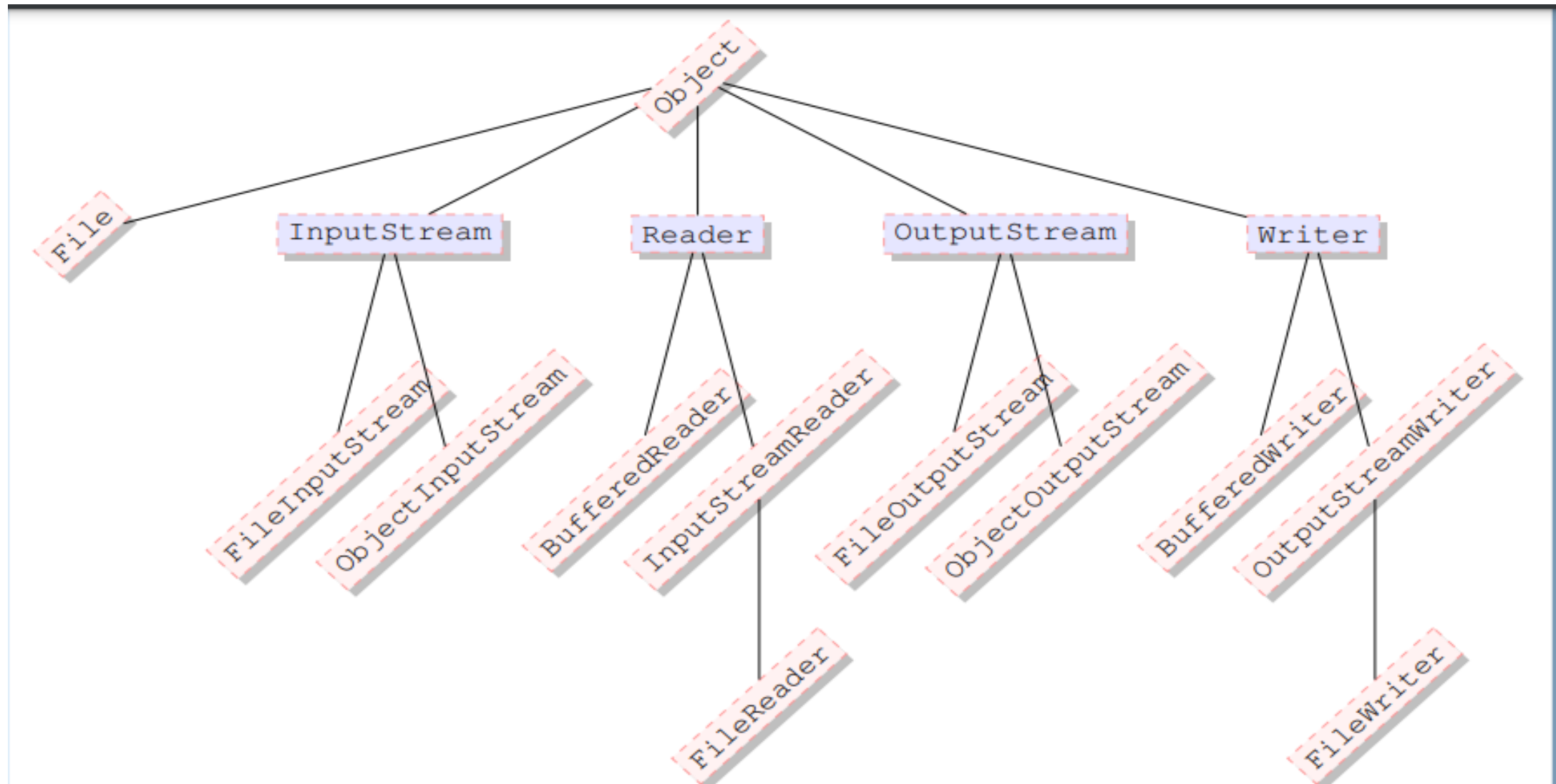
❑ **Java définit des flux** (stream en anglais) **pour lire ou écrire des données.**

❑ **En Java, les flux peuvent être divisés en plusieurs catégories :**

- les flux d'entrée (input stream) et les flux de sortie (output stream)
- les flux de traitement de caractères et les flux de traitement d'octets

❑ **Java définit aussi des classes** qui permettent de faire des traitements sur les données du flux. Toutes ces classes sont regroupées dans **le package java.io.**

le package java.io



le package java.io

	Flux d'octets	Flux de caractères
Flux d'entrée	InputStream	Reader
Flux de sortie	OutputStream	Writer

Il existe donc quatre hiérarchies de classes qui encapsulent des types de flux particuliers. Ces classes peuvent être séparées en deux séries de deux catégories différentes :

- les classes de lecture et d'écriture
- les classes permettant la lecture de caractères ou d'octets.

Les quatre classes dans le tableau sont abstraites (elles ne peuvent pas être utilisées directement); elles contiennent plusieurs sous-classes

Les Classes de package java.io

InputStream	Lecture de flots d'octets
OutputStream	Écriture de flots d'octets
Reader	Lecture de flots de caractères
Writer	Écriture de flots de caractères
File	Fichiers et répertoires
StreamTokenizer	Analyse lexicale

- les sous-classes de Reader sont des types de flux en lecture sur des ensembles de caractères
- les sous-classes de Writer sont des types de flux en écriture sur des ensembles de caractères
- les sous-classes de InputStream sont des types de flux en lecture sur des ensembles d'octets
- les sous-classes de OutputStream sont des types de flux en écriture sur des ensembles d'octets

La classe Files

la classe File du package java.io encapsule les fichiers et les répertoires. Il n'existe pas de classe pour traiter les répertoires car ils sont considérés comme des fichiers.

Une instance de la classe File est une représentation logique d'un fichier ou d'un répertoire qui peut ne pas exister physiquement sur le disque.

Constructeurs

- **File(File **d**, String **n**)**
d est le dossier du fichier, **n** est son nom.
- **File(String **n**)**
n est le nom du fichier.

Constructeurs et méthodes de la Classe `java.io.File`

Si le fichier ou le répertoire existe, de nombreuses méthodes de la classe `File` permettent d'obtenir des informations sur le fichier. Sinon plusieurs méthodes permettent de créer des fichiers ou des répertoires.

Voici une liste des principales méthodes

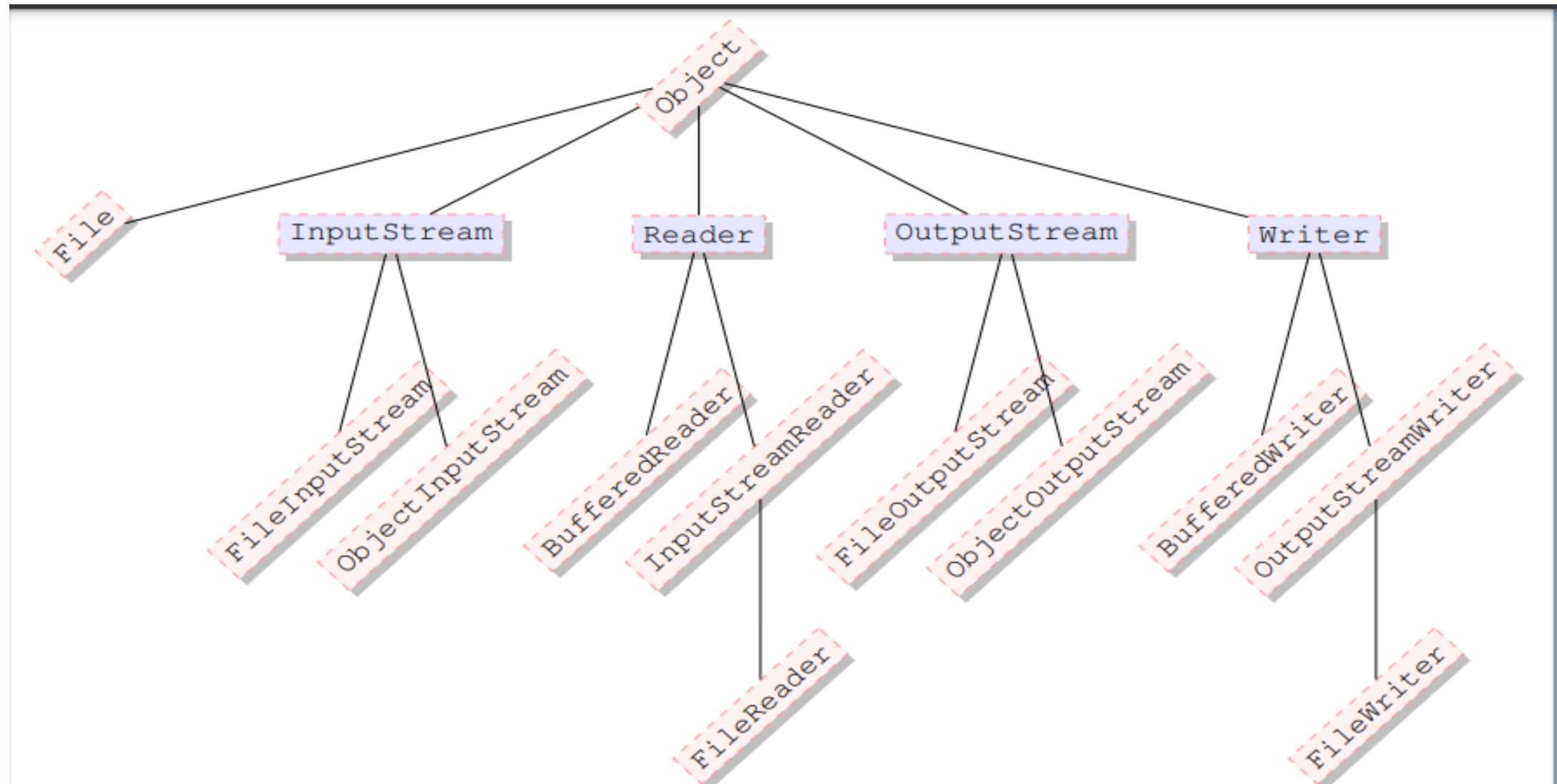
- **`boolean exists()`**
indique si le fichier existe.
- **`String getCanonicalPath()`**
renvoie le chemin complet d'accès au fichier sous forme canonique (sans `.` ou `..`).
- **`String getName()`**
renvoie le nom du fichier.
- **`boolean isDirectory()`**
indique s'il s'agit d'un dossier
- **`boolean isFile()`**
indique s'il s'agit d'un vrai fichier
- **`int length()`**
renvoie la taille du fichier
- **`String[] list()`**
renvoie la liste des fichiers d'un dossier sous forme de tableau de chaînes.

Exemple : Ecrire un programme qui affiche le contenu d'un dossier

```
public class AfficheContenu {  
    public static void main(String[] args)  
    {  
        File f =new File("c:"); // 1 on déclarer un objet de type File et on a donnée comme paramètre le dossier c:  
        String[] tab=f.list(); //2  
        File f1=null;  
        for(String t:tab)    {  
            f1 =new File("c:"+t);  
            if( f1.isDirectory()) // 3  
                System.out.println(« Dossier : "+t); //4  
            else  
                System.out.println("Fichier : "+t+" "+t.length()); // 5 }  
        }  
    }  
}
```

- 1) Le dossier de départ est nommé "c: " .
- 2) La liste des fichiers contenus dans ce dossier est obtenue par la méthode **list** sous forme de tableau de chaînes **tab**.
- 3) Les fichiers représentant un sous dossier (méthode **isDirectory()**
4) sont affichés précédés du mot dossier,
- 5) les autres sont affichés précédés du mot fichier et suivi par leur taille (méthode **length**).

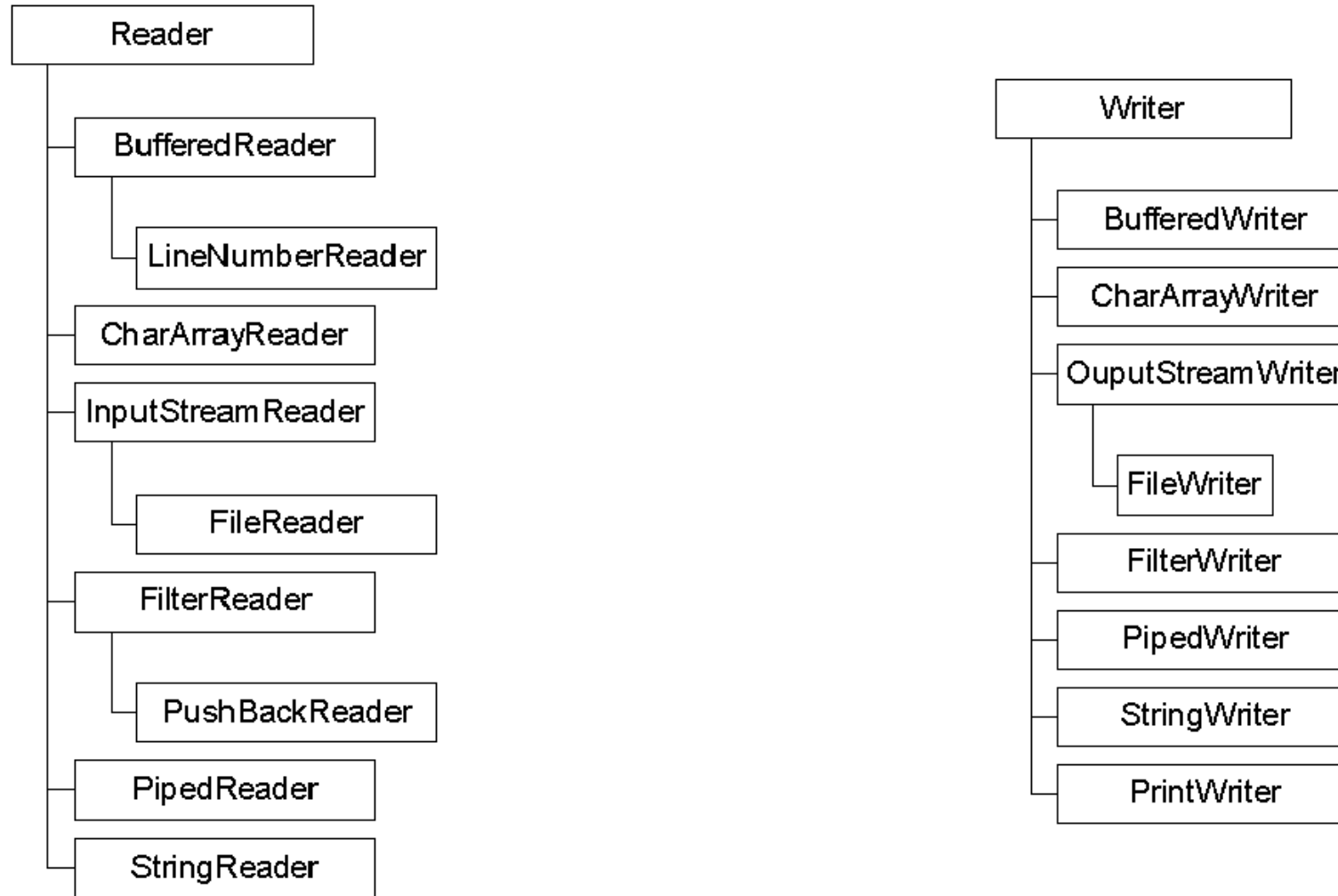
Le Package java.io.



Flux de caractères

Flux de Caractères , Types de classes

Les classes qui gèrent les flux de caractères héritent d'une des deux classes abstraites Reader ou Writer. Il existe de nombreuses sous-classes pour traiter les flux de caractères.



La classe Reader

C'est une classe abstraite qui est la classe mère de toutes les classes qui gèrent des flux de caractères en lecture.
Cette classe définit plusieurs méthodes :

<code>close()</code>	ferme le flux et libère les ressources qui lui étaient associées
<code>int read()</code>	renvoie le caractère lu ou -1 si la fin du flux est atteinte.
<code>int read(char[])</code>	lire plusieurs caractères et les mettre dans un tableau de caractères
<code>int read(char[], int, int)</code>	lire plusieurs caractères. Elle attend en paramètre : un tableau de caractères qui contiendra les caractères lus, l'indice du premier élément du tableau qui recevra le premier caractère et le nombre de caractères à lire. Elle renvoie le nombre de caractères lus ou -1 si aucun caractère n'a été lu. Le tableau de caractères contient les caractères lus.
<code>long skip(long)</code>	saute autant de caractères dans le flux que la valeur fournie en paramètre. Elle renvoie le nombre de caractères sautés.

la classe FileReader

- **la classe FileReader :**
 - est un flot (flux) pour la lecture des caractères
 - permet d'accéder à des fichiers sur disque
 - instancie et ouvre en lecture un flot à partir d'un objet File
 - méthodes :
 - long length nombre de caractères du flot
 - int read() lit un caractère sur le flot
 - int read(char[], ...) essaye de lire plusieurs caractères sur le flot.
 - close ferme le flot
- l'instruction de contrôle try ..catch est obligatoire, car la méthode read peut lever une exception

Example : FileReader

```
public class FileReader {  
    public static void main(String[] args) throws FileNotFoundException, IOException  
    {  
        File f = new File("E:/files/noms.txt");  
  
        FileReader fr = new FileReader(f);  
        int c;  
        while((c=fr.read())!=-1)  
        {  
            System.out.println(" "+c +" "+ (char)c);  
        }  
        fr.close();  
    }  
}
```

La classe **BufferedReader**

Pour améliorer les performances des flux sur un fichier, la mise en tampon des données lues ou écrites permet de traiter un ensemble de caractères représentant une ligne plutôt que de traiter les données caractères par caractères. Le nombre d'opérations est ainsi réduit.

Les classes `BufferedReader` et `BufferedWriter` permettent de gérer des flux de caractères tamponnés avec des fichiers.

- La classe Java **BufferedReader** lit le texte à partir d'un flux d'entrée de caractères, mettant en mémoire tampon les caractères lus pour permettre une lecture efficace des caractères, des tableaux et des chaînes.

Les **Méthodes**

- `close()` // close the stream
- `mark(int readAheadLimit)` // mark the position in the stream
- `markSupported()` // does it support thread tagging
- `int read()` // read buffer
- `int read(char[] cbuf, int off, int len)` // read buffer
- `String readLine()` // next line
- `boolean ready()` // can the thread
- `read reset()` // reset stream
- `skip(long n)` // skip characters

Exemple : BufferedReader

Ecrire un programme en Java permettant de lire les nombres à partir d'un fichier somme.txt et les affiche à l'écran.

```
public class BuffredReader {

    public static void main(String[] args) throws FileNotFoundException, IOException
    {
        File f =new File("E:/files/somme.txt");
        FileReader fr =new FileReader(f);
        BufferedReader br = new BufferedReader(fr);
        String s;
        Float somme=0.0f;

        while((s=br.readLine())!= null)
        {float nb=0.0f;
            String[] t=s.split(" ");
            int n=t.length;
            for(int i=0;i<n;i++)
            {
nb=Float.parseFloat(t[i]);// convertir une chaine en un float
            somme=somme+nb;        }
        }
        System.out.println("la somme est "+somme);

    }
}
```


La classe Writer

C'est une classe abstraite qui est la classe mère de toutes les classes qui gèrent des flux de caractères en écriture. Cette classe définit plusieurs méthodes :

Méthodes	Rôles
<code>close()</code>	ferme le flux et libère les ressources qui lui étaient associées
<code>write(int)</code>	écrire le caractère en paramètre dans le flux.
<code>write(char[])</code>	écrire le tableau de caractères en paramètre dans le flux.
<code>write(char[], int, int)</code>	écrire plusieurs caractères. Elle attend en paramètres : un tableau de caractères, l'indice du premier caractère et le nombre de caractères à écrire.
<code>write(String)</code>	écrire la chaîne de caractères en paramètre dans le flux
<code>write(String, int, int)</code>	écrire une portion d'une chaîne de caractères. Elle attend en paramètre : une chaîne de caractères, l'indice du premier caractère et le nombre de caractères à écrire.

Exemple : **FileWriter**

Héritage

```
java.lang.Object
    java.io.Writer
        java.io.OutputStreamWriter
            java.io.PrintWriter
```

Il faut instancier un objet de la classe `FileWriter` qui hérite de la classe `OutputStreamWriter`.
Cette classe possède plusieurs constructeurs :

Constructeur

`FileWriter(String)`

`FileWriter(File)`

`FileWriter(String, boolean)`

Rôle

Si le nom du fichier précisé n'existe pas alors le fichier sera créé. S'il existe et qu'il contient des données celles-ci seront écrasées

Idem mais le fichier est précisé avec un objet de la classe `File`

Le booléen permet de préciser si les données seront ajoutées au fichier (valeur `true`) ou écraseront les données existantes (valeur `false`)

Il existe plusieurs méthodes de la classe **FileWriter** héritées de la classe **Writer** qui permettent d'écrire un ou plusieurs caractères dans le flux.

Une fois les traitements sur le flux terminés, il faut libérer les ressources qui lui sont allouées en utilisant la méthode `close()`.

Exemple : FileReader et FileWriter

```
public class FileReaderWriter {  
    public static void main(String[] args) throws FileNotFoundException, IOException  
    {  
        File fe =new File("E:/files/entree.txt");  
  
        FileReader fr=new FileReader(fe);  
  
        File fs =new File("E:/files/sortie.txt");  
  
        FileWriter fw=new FileWriter(fs);  
        int c;  
  
        while((c=fr.read())!=-1)  
        {  
            fw.write(c);  
            System.out.println(" "+c +" "+ (char)c);  
        }  
        fr.close();  
        fw.close();  
    }  
}
```

La classe : **BufferedWriter**

La classe Java `BufferedWriter` écrit du texte dans le flux de sortie de caractères, mettant en mémoire tampon les caractères écrits pour permettre une écriture efficace de caractères, de tableaux et de chaînes. Vous pouvez spécifier la taille du tampon comme deuxième paramètre du constructeur.

Il faut instancier un objet de la classe `BufferedWriter`. Cette classe possède plusieurs constructeurs :

Constructeur	Rôle
<code>BufferedWriter(Writer)</code>	le paramètre fourni doit correspondre au flux dans lequel les données sont écrites.
<code>BufferedWriter(Writer, int)</code>	l'entier en paramètre permet de préciser la taille du buffer. Il doit être positif sinon une exception <code>IllegalArgumentException</code> est levée.

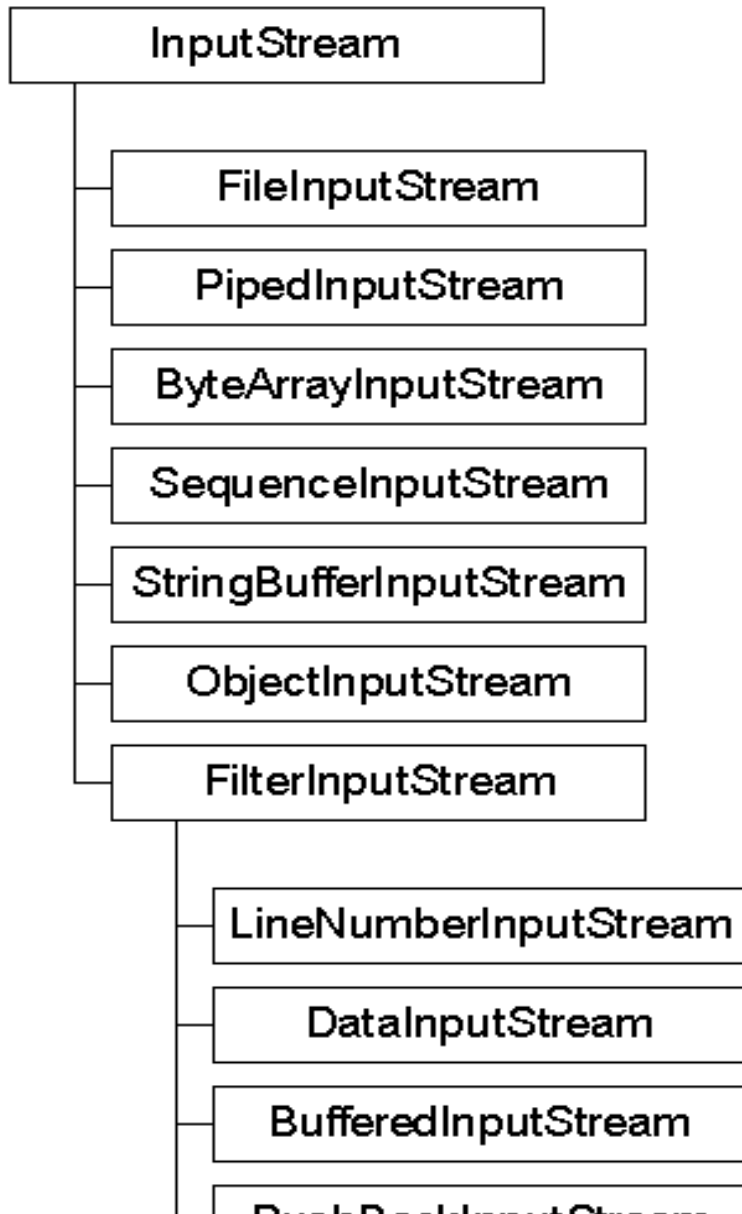
Il existe plusieurs méthodes de la classe `BufferedWriter` héritées de la classe `Writer` qui permettent de lire un ou plusieurs caractères dans le flux.

La classe `BufferedWriter` possède plusieurs méthodes pour gérer le flux :

Méthode	Rôle
<code>flush()</code>	vide le tampon en écrivant les données dans le flux.
<code>newLine()</code>	écrire un séparateur de ligne dans le flux

Les flux d'octets

La classe InputStream



Classe abstraite

- Superclasse de toutes les classes permettant la lecture de flux d'octets
- Toutes les classes manipulant un flot d'octets seront vues comme un `InputStream`
- Équivalent à une interface
- Possède un constructeur sans paramètre

La classe `InputStream`

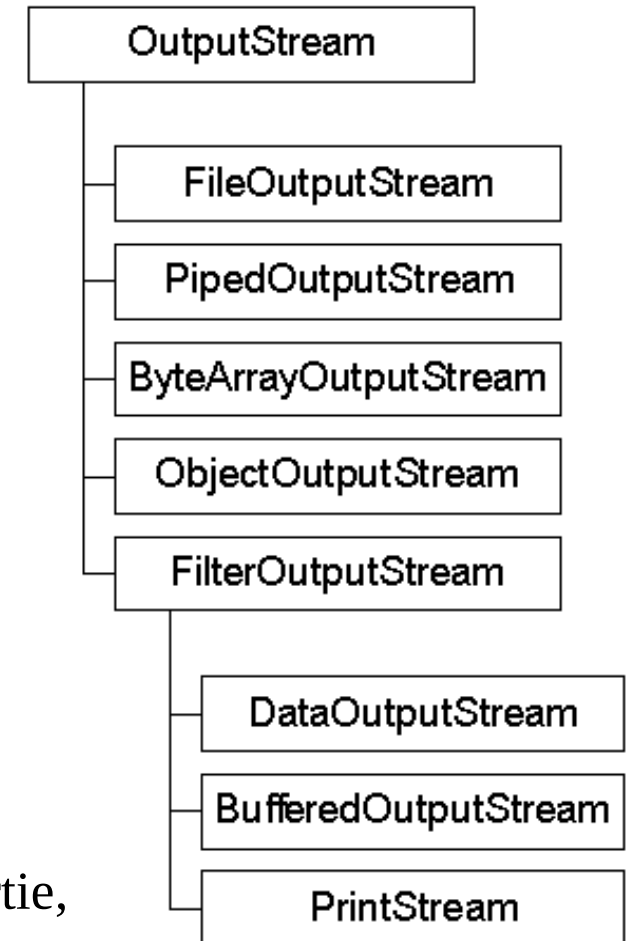
- Les méthodes principales qui peuvent être utilisées sur un `InputStream` sont :
- **`public abstract int read () throws IOException`** qui retourne l'octet lu ou -1 si la fin de la source de données est atteinte. C'est cette méthode qui doit être définie dans les sous-classes concrètes et qui est utilisée par les autres méthodes définies dans la classe `InputStream`.
- **`int read (byte[ ] b)`** qui remplit un tableau d'octets et retourne le nombre d'octets lus
- **`int read (byte [] b, int off, int len)`** qui emplit un tableau d'octets à partir d'une position donnée et sur une longueur donnée.
- **`void close ()`** qui permet de fermer un flux,
 - Il faut fermer les flux dès qu'on a fini de les utiliser. En effet, un flux ouvert consomme des ressources du système d'exploitation qui sont en nombre limité.

La classe InputStream (3)

- **Les méthodes principales qui peuvent être utilisées sur un InputStream** sont (suite) :
 - **int available ()** qui retourne le nombre d'octets prêts à être lus dans le flux,
 - **long skip (long n)** qui permet d'ignorer un certain nombre d'octets en provenance du flot. Cette fonction renvoie le nombre d'octets effectivement ignorés.

La classe **OutputStream**

- Un OutputStream est un flux d'écriture d'octets.
- La classe OutputStream est abstraite.
- Les méthodes principales qui peuvent être utilisées sur un OutputStream sont :
 - **public abstract void write (int) throws IOException**
 - qui écrit l'octet passé en paramètre,
 - **void write (byte[] b)**
 - qui écrit les octets lus depuis un tableau d'octets,
 - **void write (byte [] b, int off, int len)**
 - qui écrit les octets lus depuis un tableau d'octets à partir d'une position donnée et sur une longueur donnée,
 - **void close ()**
 - qui permet de fermer le flux après avoir éventuellement vidé le tampon de sortie,
 - **flush ()** qui permet de purger le tampon en cas d'écritures bufférisées.



Les classes `FileInputStream` et `FileOutputStream`

- Le **`FileInputStream`** ;lire des octets à partir d'un fichier. cette classe dérive de la **`InputStream`** classe abstraite
- Il faut instancier un objet de la classe **`FileInputStream`**. Cette classe possède plusieurs constructeurs qui peuvent tous lever l'exception **`FileNotFoundException`**.

Constructeur	Rôle
<code>FileInputStream(String)</code>	Ouvre un flux en lecture sur le fichier dont le nom est donné en paramètre
<code>FileInputStream(File)</code>	Idem mais le fichier est précisé avec un objet de type <code>File</code>

- Le but principal de la classe **`FileOutputStream`** est d'écrire des octets dans un fichier.
- **`FileOutputStream`** est l'une des implémentations de la `OutputStream` classe abstraite. Possède plusieurs constructeurs qui peuvent tous lever l'exception **`IOException`**.

Constructeur	Rôle
<code>FileOutputStream(String)</code>	Si le fichier précisé n'existe pas, il sera créé. Si il existe et qu'il contient des données celles-ci seront écrasées.
<code>FileOutputStream(String, boolean)</code>	Le booléen permet de préciser si les données seront ajoutées au fichier (valeur <code>true</code>) ou écraseront les données existantes (valeur <code>false</code>)

Exemple FileOutputStream

```
package demoio;
import java.io.File;
import java.io.FileOutputStream;
import java.io.IOException;
public class DemoIO {
    public static void main(String[] args) {
        File f = new File("exemple.txt");
        try {
            FileOutputStream Fo = new FileOutputStream(f);
            byte[] b = new byte[5];
            b[0] = 115;
            b[1] = 65;
            b[2] = 45;
            b[4] = 110;
            Fo.write(b);
            Fo.close();
        } catch (IOException ex) {
            System.out.println("impossible de créer le fichier");
        }
    }
}
```

Exemple FileInputStream

```
package demoio;
import java.io.File;
import java.io.FileInputStream;
import java.io.IOException;
public class DemoIO {
    public static void main(String[] args) {
        File f = new File("exemple.txt");
        try {
            FileInputStream FI =new FileInputStream(f);
            byte[] b=new byte[5];
            int r = FI.read(b);
            System.out.println("les octets lu "+b.toString());
            FI.close();
        } catch (IOException ex) {
            System.out.println("impossible de créer le fichier");
        }

    }
}
```

Exemple 1:La classe FileInputStream

- Le FileInputStream permet de lire des octets à partir d'un fichier. Cette classe dérive de la InputStream classe abstraite. Nous allons écrire quelques lignes de texte dans notre fichier " **note.txt** "

```
public class Main {  
    public static void main(String[] args) throws FileNotFoundException.  
    {  
        FileInputStream fis= new FileInputStream(" E:\\files\\note.txt");  
        int i;  
        while ((i=fis.read()) != -1) //int read() Retourne le prochain octet dans le flot de données et Retourne -1 si fin de flot atteinte  
        { System.out.print((char)i); }  
  
    } }
```

Exemple 2: La classe FileOutputStream

Le but principal de la `FileOutputStream` classe est d'écrire des octets dans un fichier.

```
public class Main { public static void main(String[] args) throws IOException
{
    File f = new File("E:\\files\\test.txt");
    FileOutputStream fos = new FileOutputStream(f);
    String s = "Hi! Welcome to ISET!";
    fos.write(s.getBytes());
    f.close();

}
}
```

Lors de la création de `File` objet, nous avons passé le chemin souhaité au constructeur. Nous n'avons pas besoin de le créer à l'avance : s'il n'existe pas, le programme le créera

Exemple 3: La classe FileOutputStream

Vous pouvez également vous débrouiller sans créer d'objet supplémentaire, en passant simplement une chaîne avec le chemin .

```
public class Main { public static void main(String[] args) throws IOException
{
    FileOutputStream fos = new FileOutputStream ("E:\files\test.txt");
    String s = "Hi! Welcome to ISET!";
    fos.write(s.getBytes());
    fos.close();
}}
```

Exemple 4: La classe FileOutputStream

- Si on exécute le code de l'exemple Précédent plusieurs fois de suite., on trouve dans le fichier une ligne seulement
- Si nous voulons écrire notre message d'accueil dans un fichier trois fois de suite nous avons besoin dans chaque cas,

le FileOutputStream constructeur peut prendre un paramètre supplémentaire boolean append.

`FileOutputStream(String, boolean)` le boolean permet de préciser si les données seront ajoutées au fichier(valeur true) ou écraseront les données existantes (valeur false).

```
public class Main { public static void main(String[] args) throws IOException {  
    FileOutputStream fos = new FileOutputStream("E:\\files\\test.txt", true);  
    String s = "Hi! Welcome to ISET";  
    fos.write(s.getBytes());  
    fos.close(); //n'oubliez pas d'utiliser la méthode close() pour libérer des  
                //ressources.  
}}
```


Exemple 5: Les classes FileInputStream et FileOutputStream

```
public class Main {  
    public static void main(String[] args) throws FileNotFoundException.  
{  
    FileInputStream fis= new FileInputStream(" E:\files\note.txt");  
  
    FileOutputStream fos= new FileOutputStream(" E:\files\Resultat.txt");  
  
    int i;  
    while ((i=fis.read()) != -1) //int read() Retourne le prochain octet dans le flot de données et Retourne -1 si fin de flot atteinte  
  
    {fos.write(i); } // écrire dans le fichier resultat  
  
    fos.close();  
    fis.close();  
  
} }
```

BufferedInputStream

- Les flux tamponnés sont principalement nécessaires pour optimiser les E/S.
- Accéder à une source de données, comme la lecture d'un fichier, est une opération coûteuse en termes de performances Et accéder à un fichier pour lire chaque octet est du gaspillage.
- C'est pourquoi **BufferedInputStream** lit les données non pas un octet à la fois, mais **par blocs**, et les stocke temporairement dans un tampon spécial. Cela nous permet d'optimiser le programme en réduisant le nombre de fois que nous accédons au fichier.
- Cette classe possède plusieurs constructeurs qui peuvent tous lever une exception de type **FileNotFoundException**:

Constructeur	Rôle
BufferedReader(Reader)	le paramètre fourni doit correspondre au flux à lire.
BufferedReader(Reader, int)	l'entier en paramètre permet de préciser la taille du buffer. Il doit être positif sinon une exception de type <code>IllegalArgumentException</code> est levée.

```
public class Main {  
    public static void main(String[] args) throws IOException {  
        FileInputStream fis = new FileInputStream("E:\\files\\test.txt");  
        BufferedInputStream bfis = new BufferedInputStream(fis, 200);    //lit 200 octets à la fois  
        int i;  
        while((i = bfis .read()) != -1)  
        { System.out.print((char)i); }  
    } }  
}
```

La classe StringTokenizer

- ❑ La classe **StringTokenizer** fait partie du package **Java.util** et fournit un moyen simple de diviser une chaîne en jetons.
- ❑ La classe a deux constructeurs:
 - ❑ un qui prend une chaîne à tokeniser et un caractère ou une chaîne délimiteur,
 - ❑ un qui prend les mêmes arguments ainsi qu'un indicateur booléen indiquant s'il faut ou non inclure le délimiteur en tant que jeton.
- ❑ La classe **StringTokenizer** fournit des méthodes pour parcourir les jetons et effectuer diverses opérations sur eux:
 - ❑ **tokenizer** String pour diviser une chaîne en jetons en fonction d'un délimiteur spécifié.
 - ❑ **nextToken()** récupérer chaque jeton tour à tour,
 - ❑ **hasMoreTokens()** vérifier s'il reste d'autres jetons.
 - ❑ **length()** utiliser pour obtenir la longueur de chaque jeton.
- ❑ La chaîne du tokenizer peut être n'importe quelle chaîne qui sépare les jetons, telle qu'une virgule, un point-virgule ou un espace.

Exemple : StringTokenizer

String chaine= "Ali Dridi L2DSI2"

diviser cette chaîne en mots individuels, utiliser le caractère espace comme délimiteur, :

```
StringTokenizer ch_a_diviser = new StringTokenizer(chaine, " ");  
while (ch_a_diviser.hasMoreTokens())  
{ String t = ch_a_diviser.nextToken();  
  System.out.println(t); }
```